

caloryx-service

High-Level Overview

What the backend does, how the pieces fit together, and the mental model you need before reading any code.

Who this is for. A developer who knows how backends work in general, but has never written Python or Django. Every framework-specific idea is introduced in plain terms and mapped to the equivalent you already know from the Node/Express/NestJS world. Read this document first; the companion *Deep Dive* then walks through every service file by file.

Repository [caloryx-service](#) · **Branch** [feature/meal-logging-service](#)

Stack [Django](#) + [Django REST Framework](#) · [Prisma \(prisma-client-py\)](#) · [PostgreSQL](#) · [Clerk](#) · [OpenAI](#)

Companion document [CaloryX-Service-Deep-Dive.pdf](#)

1 What this service is

One Django project that serves a calorie-tracking mobile app. It does three jobs.

1 · Identity

Anonymous "guest" sessions and real sign-ins (Google / Apple, via Clerk), plus the migration of a guest's data onto a real account once they sign up.

2 · Onboarding & the plan

Collects height, weight, target weight, activity level and date of birth, then computes the authoritative daily calorie + macro target and stores it.

3 · Meal logging

Two ways to record what you ate: pick foods explicitly from a catalog, or type a sentence ("200g rice and 2 rotis") and let an AI-assisted pipeline turn it into a reviewable draft.

Everything else is support

Cost control for the AI calls, caching, idempotency for retries, telemetry, privacy/retention, and health probes — all of which exist because job 3 talks to a paid external model.

The one principle that explains most design decisions

CORE RULE

The language model never produces a number. It is only ever allowed to turn free text into structured intent — "the user said *rice*, quantity *200*, unit *g*". Every calorie and macro figure is then derived deterministically from the food catalog by pure Python code. The model's response schema literally has nowhere to put a calorie value.

Once you internalise that, a lot of the codebase reads naturally: there are two "pure" calculation packages with no database access at all (`engine/` and `nutrition/`), a pure text package (`chatparser/`), and everything AI-related is isolated behind a single thin wrapper (`llm/`) so the provider could be swapped by editing one file.

Two feature waves in one codebase

WAVE	WHAT IT ADDED	LIVES IN
Onboarding shipped first	Auth, profile capture, the calorie/macro engine, the plan screen.	<code>authx/</code> <code>onboarding/</code> <code>engine/</code>
AI Meal Assistant built in 8 chunks	Food catalog, manual meal logging, editable drafts, the free-text chat pipeline, AI escalation, quota, caching, wellbeing safeguards, telemetry.	<code>meals/</code> <code>nutrition/</code> <code>assistant/</code> <code>chatparser/</code> <code>llm/</code>

2 Stack primer for a non-Python developer

Five unfamiliar names, each with a direct equivalent you already know.

IN THIS REPO	WHAT IT ACTUALLY IS	CLOSEST THING YOU ALREADY KNOW
Django	The web framework. Provides settings, URL routing, middleware, and a CLI (<code>manage.py</code>).	Express/NestJS + its config and CLI. Note: Django's own ORM and admin are deliberately unused here.
Django REST Framework (DRF)	The API layer on top of Django: class-based views, request validation, authentication classes, permissions, throttling.	A controller + DTO-validation + guards layer, e.g. NestJS controllers with <code>class-validator</code> .
Serializer	A DRF class that validates an incoming JSON body and/or shapes an outgoing one.	A Zod / Joi schema for input, plus a response DTO mapper for output. One class does both jobs.
Prisma (prisma-client-py)	The same Prisma you know, with a Python client generated from <code>prisma/schema.prisma</code> .	Identical to Prisma in Node — same schema file, same <code>db push / migrate</code> commands.
Clerk	Hosted auth. The mobile app signs in with Clerk on the device; this service only <i>verifies</i> the resulting JWT.	Auth0 / Firebase Auth. No Google or Apple secret ever lives in this repo.

PYTHON CONVENTIONS YOU WILL BUMP INTO IMMEDIATELY

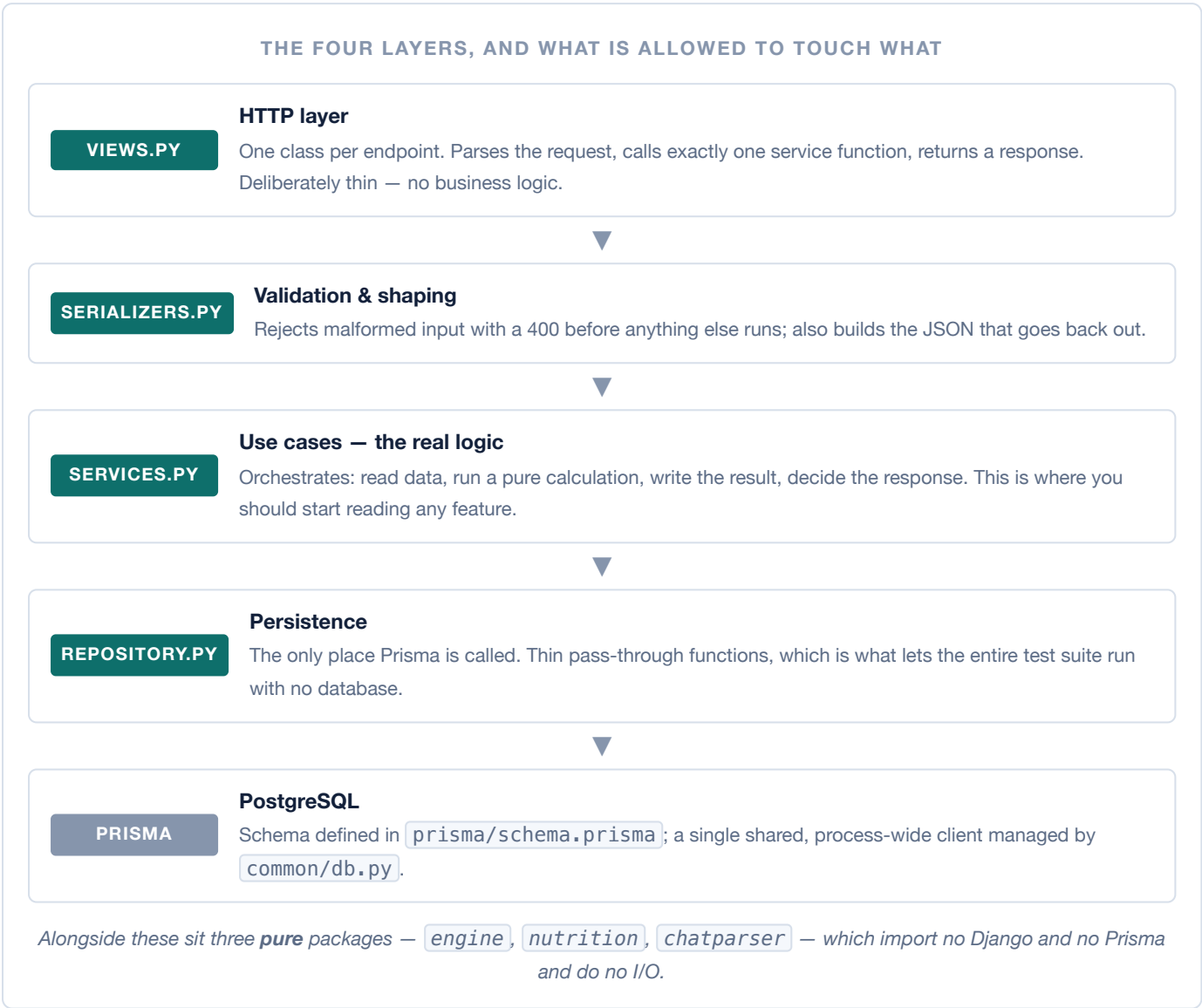
Leading underscore	<code>_helper()</code> means "private to this module, don't import it elsewhere". It is a convention, not enforced.
<code>__init__.py</code>	A folder's entry point — the equivalent of <code>index.ts</code> . It re-exports the package's public surface.
Type hints	<code>def f(x: str) -> Optional[int]</code> is TypeScript's <code>(x: string) => number null</code> . They are documentation, not runtime checks.
<code>@dataclass(frozen=True)</code>	An immutable value object — a <code>readonly</code> struct with an auto-generated constructor.
Decorators <code>@something</code>	Same idea as TypeScript decorators: wrap or annotate the function/class below.
Docstrings	The triple-quoted block at the top of a file or function. In this codebase they are unusually thorough — they carry the <i>reasoning</i> , and are worth reading before the code.

WORTH KNOWING

`DATABASES = {}` in `caloryx/settings.py` is deliberate: Django's ORM is switched off entirely so nothing can accidentally bypass Prisma. There are no Django models and no Django migrations in this project — the schema lives only in `prisma/schema.prisma`.

3 How the code is organised

Every feature folder follows the same four-layer shape. Learn it once and every app reads the same way.



The folder map

FOLDER	ROLE	WHY IT EXISTS AS ITS OWN THING
caloryx/	Django project config	Settings, URL table, WSGI/ASGI entry points. Every tunable constant in the system is declared here, read from env vars.
common/	Cross-cutting plumbing	Prisma client lifecycle, the single error envelope, request-id middleware, health probes, startup checks.
authx/	Identity	Verifies Clerk JWTs, mints guest tokens, handles guest→account claiming. Isolated so no other module ever thinks about tokens.
engine/	PURE Calorie & macro maths	BMR → TDEE → target → macros. No Django, no DB, no I/O — so it is trivially testable and could be lifted into its own service without touching callers.

FOLDER	ROLE	WHY IT EXISTS AS ITS OWN THING
onboarding/	Profile & plan endpoints	The only consumer of <code>engine/</code> . Combines the pure calculation with persistence.
nutrition/	PURE Food maths	per-100g × grams ÷ 100, raw↔cooked yield conversion, unit→grams resolution. Same purity rule as <code>engine/</code> .
meals/	Food catalog & logged meals	Manual, fully-quantified logging with no chat and no AI. The assistant is built directly on top of it rather than duplicating it.
chatparser/	PURE Text understanding	Normalisation, keyword pre-classification, the regex grammar, food-match scoring, PII redaction. Turns text into structured intent; never touches the database.
llm/	Model provider wrapper	The only file that imports the OpenAI SDK. Holds the system prompt and the strict JSON schema the model must answer in.
assistant/	Drafts & the chat pipeline	The largest module. Owns the editable draft, the tier ladder, quota, caching, idempotency and the confirm→logged-meal handoff.
tests/	1,082 tests	None of them need a database or a real model call; repositories and the LLM client are the seams that get stubbed.

4 The lifecycle of one request

Identical for every endpoint in the service — worth committing to memory.

REQUEST PATH, FROM MOBILE APP TO DATABASE AND BACK

1 • Mobile app sends the request

Header: `Authorization: Bearer <token>`. The token is either a Clerk session JWT or a guest token this service minted.

2 • RequestIdMiddleware

Reuses an inbound `X-Request-Id` or generates one. Stored in a context variable, stamped on every log line, every AI telemetry row, and the response header.

Why

One mobile trace can be followed end-to-end across logs, chat messages and model-call records.

3 • BearerAuthentication

Reads the JWT's `alg` header: RS256 → verify against Clerk's JWKS; HS256 → verify as a guest token. Produces an `Actor` value object.

401 exit

Invalid, expired, or a guest session that has already been claimed by a real account.

4 • View + serializer

The view validates the body through a serializer. Anything malformed stops here.

400 exit

`validation_error`, with per-field details.

5 • Service function

The actual use case. Reads through the repository, runs pure calculations, writes results, assembles the response payload.

4xx exit

A `DomainError` subclass — e.g.
`profile_required` (409),
`draft_version_conflict` (409),
`age_below_minimum` (422).

6 • Repository → Prisma → Postgres

The only layer that talks to the database.

7 • Response

Success payload, or the single error envelope below. `X-Request-Id` is echoed back either way.

One error shape, everywhere

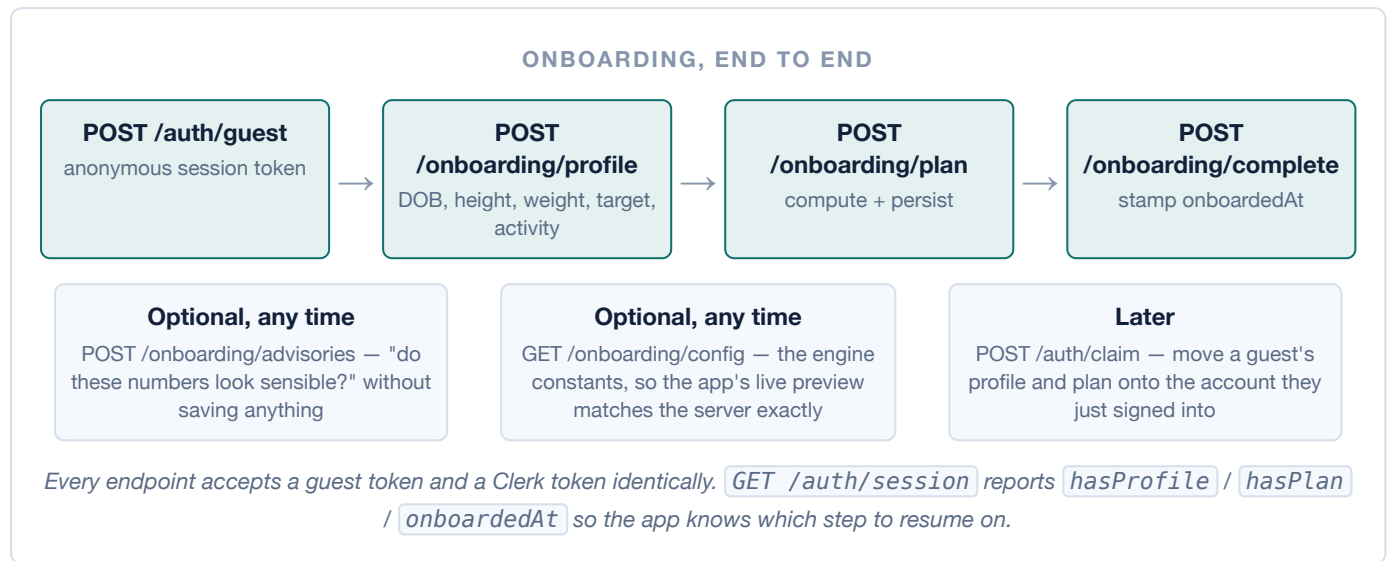
Every failure in the system — validation, domain rule, auth, or an unexpected crash — comes back in exactly this envelope, so the mobile client has a single shape to branch on:

```
{ "error": { "code": "age_below_minimum", "message": "...", "details": {}, "requestId": "..." } }
```

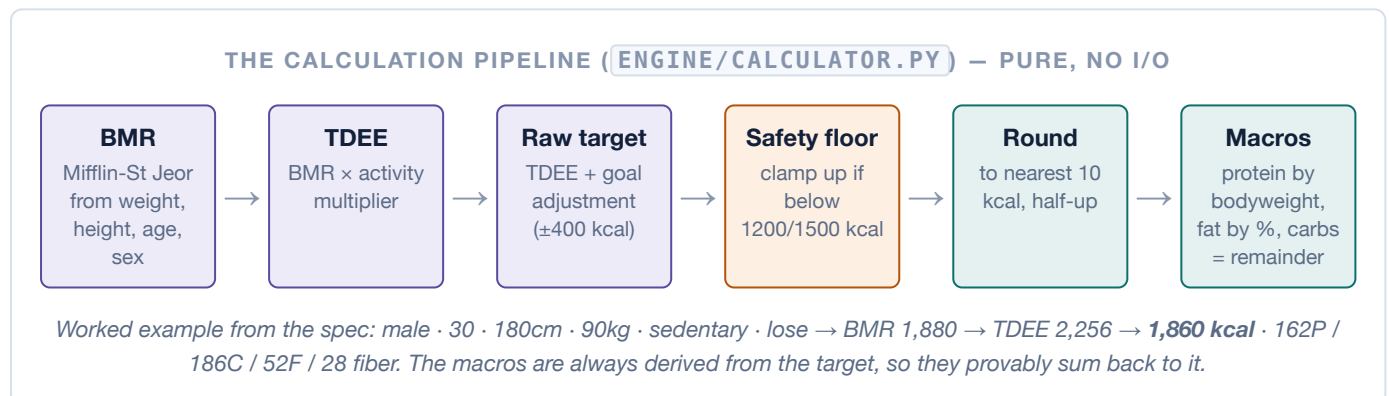
400 validation_error **401** missing/invalid token **409** profile_required · plan_required ·
claim_conflict · open_draft_exists · draft_version_conflict · idempotency_key_reused **422**
age_below_minimum · invalid_quantity · operation_stale · operation_expired

5 Journey A — onboarding a new user

From first app launch to a stored calorie target.



What the engine actually computes



TWO DETAILS THAT SURPRISE PEOPLE

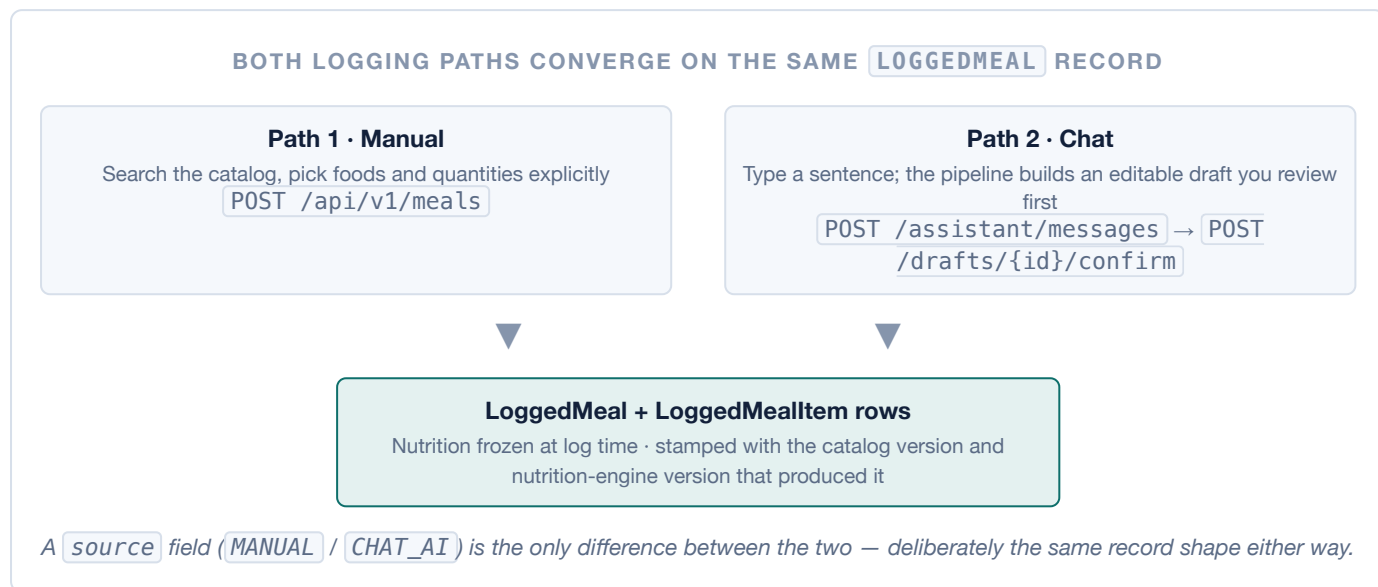
The goal is never asked for. It is derived by comparing target weight to current weight against a configurable "maintain band". **Fiber is never counted as energy** — it is a subset of carbohydrate grams, so `macroEnergyKcal` deliberately has no fiber key.

TUNABLE WITHOUT AN APP RELEASE

Activity multipliers, goal adjustments, macro ratios, safety floors and rounding all live in an `EngineConfig` database row, with compiled defaults as the fallback. Every stored plan records which config row produced it, so an old plan stays explainable after the constants are retuned.

6 Journey B — logging a meal

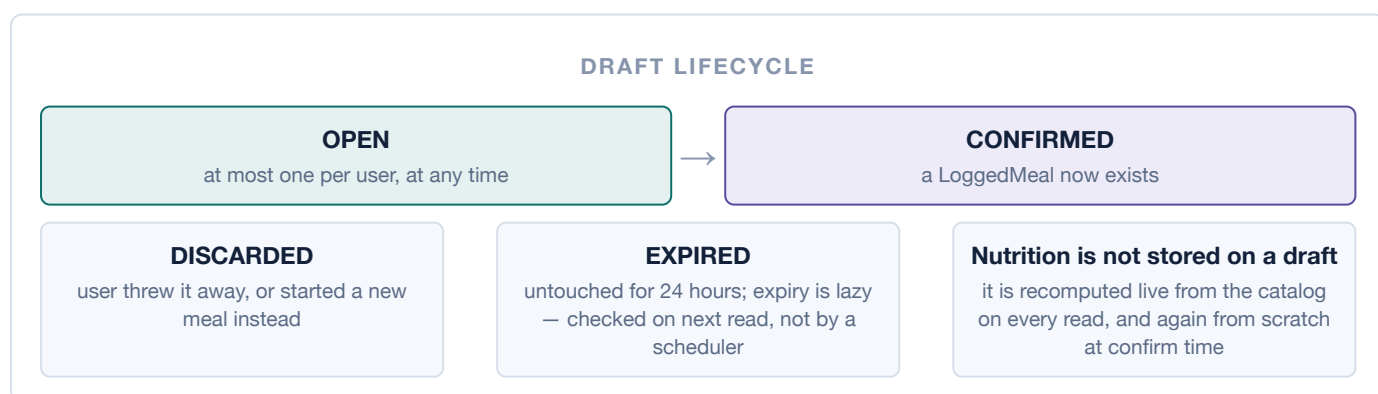
Two entry points, one destination.



Why a draft exists at all

A chat message never logs a meal directly. It creates or mutates a **MealDraft** — a live, server-side, editable object that the app renders as a preview card. The user can adjust portions, add or remove items, or change the meal slot, all through further chat messages or structured API calls, and only then tap *Confirm & Log*. This gives three things:

- **A review step** before anything enters the diary, which is what makes an imperfect parse acceptable.
- **Optimistic concurrency.** Every mutation sends the draft's `version`; a stale version is rejected with `draft_version_conflict` instead of silently overwriting a change from another device.
- **An audit trail.** Every mutation writes a `DraftOperation` row, which is what makes a future "undo" a feature rather than a rewrite.



7 The chat pipeline at a glance

The heart of the system. A ladder of tiers, cheapest first — a paid model call is the last resort, not the first move.

TIER LADDER — A MESSAGE TRIES EACH RUNG IN ORDER AND STOPS AT THE FIRST THAT SUCCEEDS

T-1 • Pre-classifier

FREE Keyword/regex check for non-logging intents: greetings, app help, "how many calories do I have left", nutrition questions, advice-seeking.

Exits here

Answers with scripted copy or a local database read. Never touches quota or a model.

T0 • Cache

FREE Have *you* sent this exact sentence before? If so, replay the stored interpretation. Then: has *anyone*? A global, PII-free cache shared across all users.

Exits here

Re-resolves through the catalog rather than replaying stored numbers, so a cache hit can never go stale against a catalog edit.

T1 • Deterministic grammar

FREE Regex: `QUANTITY + UNIT + [STATE] + FOOD`, split on connectors. "200g rice and 2 rotis" → two phrases. Also handles edit commands on an open draft.

Exits here

The common, well-formed case. Builds a draft with zero cost and zero latency.

Quota gate

GATE 20 AI parses per rolling 24 hours per user. Checked only now — everything above this line is unlimited.

Exits here

Quota exhausted → an honest message and a `quotaExceeded` flag; manual entry still works.

T2 • Small model

PAID One call, one message, the whole intent envelope. Strict JSON schema — the model cannot answer with prose.

Exits here

Usually. Handles descriptive phrasing the grammar can't reach ("a couple of eggs and some toast").

T3 • Large model

PAID A second opinion, only when T2 *succeeded* but reported low confidence — never a rescue for a T2 failure. Does not consume a second quota unit.

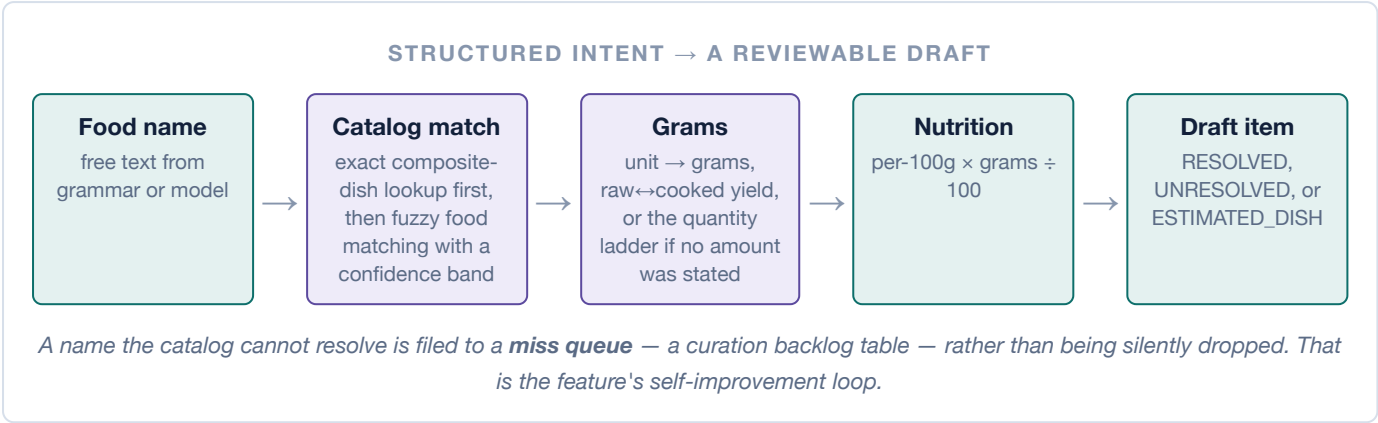
Also

Always runs when T2 flags a wellbeing concern, where a false negative matters more than the cost.

Give up honestly

"I couldn't work out what food that was." Unparsed fragments are always reported back to the user verbatim, never silently dropped.

What happens after a successful parse

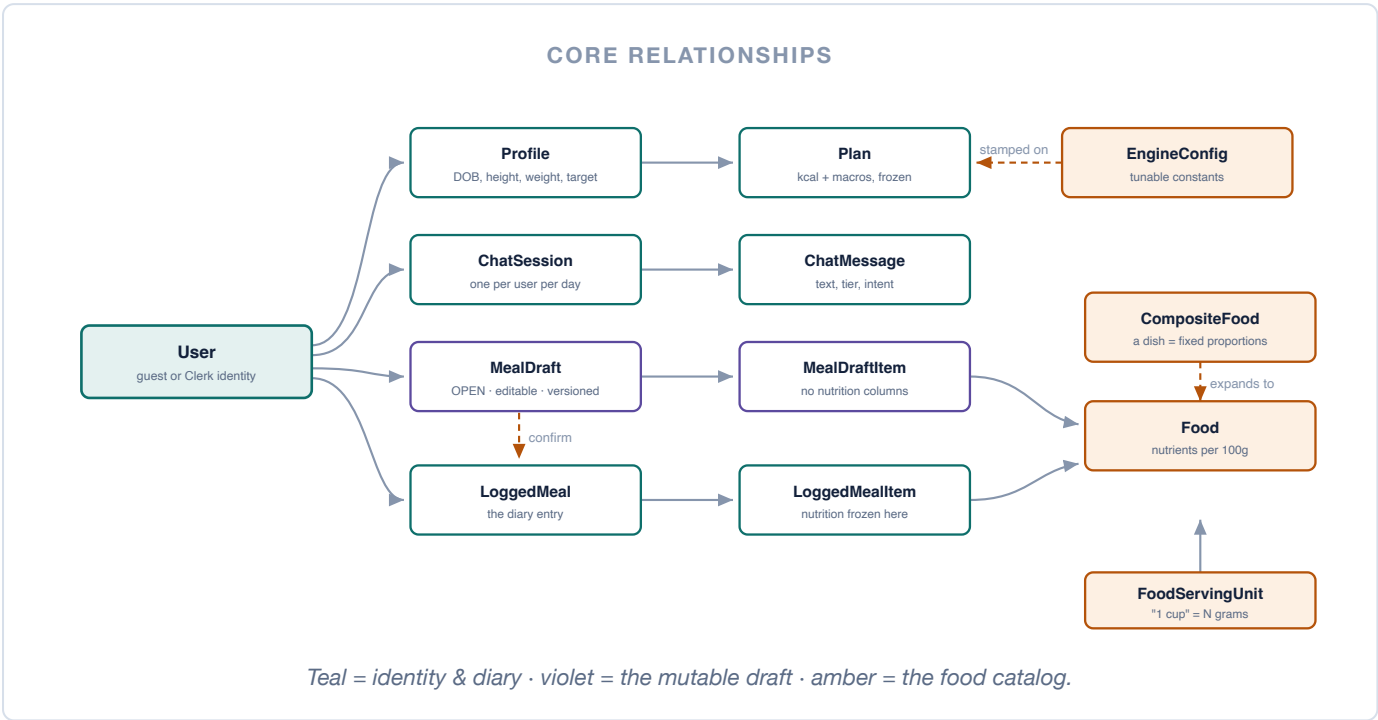


The three states a draft item can be in

RESOLUTION	MEANING	NUTRITION SHOWN
RESOLVED	Matched a real catalog food with a known gram amount.	Full calories + all macros.
UNRESOLVED	The text was understood but the catalog has nothing for it.	Nothing. Shown to the user as an item needing attention; filed to the miss queue.
ESTIMATED_DISH	A named dish with no confident ingredient breakdown ("misal pav"). The model supplies only a <i>category</i> ; a curated per-100g calorie band supplies the number.	A calorie range and midpoint. Macros are deliberately <code>null</code> — shown as "—", never guessed.

8 The data model at a glance

Twenty-odd Prisma models. These are the ones that carry the story.



The supporting tables, grouped by the problem they solve

PROBLEM	TABLES	WHAT THEY DO
Cost control	AiQuotaCounter · AiCircuitBreaker · GlobalParseCache · ParseEvent	Cap per-user AI usage; stop calling a failing provider for a cooldown window; share one paid parse of a common phrase across all users; record cost, latency and confidence of every model call.
Safe retries	IdempotencyRecord	A repeated request with the same key replays the original response instead of creating a second meal. Its TTL is deliberately derived from the offline-queue window, never tuned independently.
Getting better over time	FoodMissQueue · UserServingPreference	Every unresolvable name becomes curation backlog; every confirmed portion teaches the system this user's usual serving of that food.
Reproducibility	CatalogVersion · Draft0operation	Cache keys and logged meals are stamped with the catalog/parser/normalisation versions in force, so a data or code change invalidates stale interpretations instead of serving them.
Curation	CompositeFood · CompositeFoodComponent · DishCategoryProfile	Named dishes and their fixed component proportions, plus per-category calorie bands. All human-curated: decomposing a dish is a catalog lookup, never a language task.

9 The complete API surface

Base path `/api/v1`. Every endpoint accepts guest and signed-in sessions alike, except `/auth/claim`.

IDENTITY

METHOD	PATH	PURPOSE
POST	<code>/auth/guest</code>	Create an anonymous session; returns a guest bearer token. The only unauthenticated write, and the only throttled endpoint.
GET/POST	<code>/auth/session</code>	Resolve the current session and report where to resume onboarding.
POST	<code>/auth/claim</code>	Move a guest's data onto the signed-in account. The guest token then stops working — that 401 is the signal to switch tokens, not an error.

ONBOARDING

METHOD	PATH	PURPOSE
GET/POST	<code>/onboarding/profile</code>	Upsert or read the collected inputs. Returns non-blocking advisories alongside.
POST	<code>/onboarding/advisories</code>	Sanity-check unsaved values mid-flow. Reads nothing, writes nothing.
POST/GET	<code>/onboarding/plan</code>	Compute and persist the plan / fetch the stored one.
POST	<code>/onboarding/complete</code>	Stamp <code>onboardedAt</code> . Refuses if no plan exists.
GET	<code>/onboarding/config</code>	The active engine constants, so the app's live preview matches the server.

MEALS — MANUAL LOGGING

METHOD	PATH	PURPOSE
GET	<code>/meals/foods?q=</code>	Catalog search. An empty result is filed to the miss queue.
POST/GET	<code>/meals</code>	Log a fully-quantified meal / list this user's meals.
GET/DELETE	<code>/meals/{id}</code>	Fetch or delete one logged meal.
PATCH/DELETE	<code>/meals/{id}/items/{itemId}</code>	Adjust or remove one item; the meal's totals are recomputed from the current catalog.

ASSISTANT — DRAFTS & CHAT

METHOD	PATH	PURPOSE
POST	<code>/assistant/messages</code>	The text pipeline. Always returns 200 — it is a conversational turn, not a resource creation. Idempotent on <code>clientMessageId</code> .
POST	<code>/assistant/drafts</code>	Create a draft from structured items (no text, no AI).
GET/PATCH/DELETE	<code>/assistant/drafts/{id}</code>	Read, rename/re-slot, or discard the open draft.
POST	<code>/assistant/drafts/{id}/items</code>	Add an item by exact food id.
PATCH/DELETE	<code>/assistant/drafts/{id}/items/{itemId}</code>	Adjust portion / remove item. Every mutation carries the draft <code>version</code> .

METHOD	PATH	PURPOSE
POST	/assistant/drafts/{id}/confirm	Turn the draft into a <code>LoggedMeal</code> . Requires an idempotency key.
GET	/assistant/quota	Remaining AI parses and when the window resets. Read-only; never consumes.

OPERATIONAL

METHOD	PATH	PURPOSE
GET	/healthz · /readyz	Liveness (process is up) and readiness (database reachable).
GET	/api/schema	OpenAPI 3 document, generated from the serializers so it cannot drift. Feeds frontend type generation.
GET	/api/schema/docs · /redoc	Swagger UI / ReDoc, for humans.

10 Rules that hold across the whole system

The server is the authority on numbers

The app may render an optimistic preview, but at confirm time every item is recomputed from scratch against the *current* catalog. If that differs from what the user was shown, the response says so rather than silently disagreeing.

Nothing is silently dropped

An unparsed fragment comes back as `unconsumedText`. An unmatched food becomes an `UNRESOLVED` item and a miss-queue row. A rejected offline operation is surfaced, never quietly discarded.

Missing is not zero

An estimated dish has calories but genuinely unknown macros, so they stay `null` and render as "—". The summing logic starts from "unknown" rather than 0.0 precisely so an unknown can never be laundered into a confirmed zero.

Every mutation is retry-safe

Confirm and message-send take a client-supplied key; a repeat replays the original response. Reusing a key with *different* content is rejected — a replay must match both the key and the request.

Degrade, never 500

A model outage, a malformed model response, a missing API key, or a tripped circuit breaker all fall back to the same graceful "I couldn't work that out" reply. Chat is never fully unavailable.

Privacy by construction

Emails and phone numbers are redacted before any text leaves the process. Cache keys are hashes of normalised food text only. Raw chat messages have a 30-day retention window; logged meals do not.

Where to look when...

YOU WANT TO UNDERSTAND...	START HERE
How a chat message becomes a draft	<code>assistant/services.py</code> → <code>send_message</code> → <code>_process_message</code> → <code>_process_new_meal</code>
How a calorie target is computed	<code>engine/calculator.py</code> → <code>calculate_plan</code>
How a food's nutrition is derived	<code>nutrition/calculator.py</code> + <code>meals/services.py</code> → <code>resolve_item</code>
What the model is allowed to say	<code>llm/schema.py</code> and <code>llm/prompts.py</code>
Which tier handled a given message	The <code>tier</code> field on the response, and the <code>ChatMessage</code> / <code>ParseEvent</code> rows
Why a decision was made the way it was	The docstring at the top of the file — this codebase documents reasoning, not just behaviour
What is deliberately unfinished	<code>documents/known-issues.md</code> — the durable record of every accepted trade-off and open decision

IMPORTANT CONTEXT BEFORE YOU TRUST ANYTHING

Per `documents/known-issues.md`: this service has never run against a real PostgreSQL database or made a real OpenAI API call. The full test suite passes, but it passes against stubbed repositories and a mocked model client. Treat every "verified" claim in the codebase as *verified against mocks*. Confidence thresholds, quota limits and cost figures are honest guesses awaiting real traffic to calibrate against.